Logic for Computer Science Project : Report

Kuckart Marco

1 Question 1

For this question let $p_{i,j}$ be the proposition "the cell i is reached with j moves".

First, all cells must be visited : $\forall i \bigvee_j p_{i,j}$

It is obvious that given the definition of $p_{i,j}$, we have the following constraint : A cell cannot be reached in two different numbers of moves.

So we have

$$\forall i \, (\forall j, k, j \neq k, \neg(p_{i,j} \wedge p_{i,k}) \leftrightarrow (\neg p_{i,j} \vee \neg p_{i,k}))$$

Another intuitive constraint is that two different cells cannot be reached in the same number of moves.

So we also have

$$\forall i \, (\forall j, k, j \neq k, \neg(p_{j,i} \wedge p_{k,i}) \leftrightarrow (\neg p_{j,i} \vee \neg p_{k,i}))$$

All possible numbers of moves have to be assigned, but this is a logical consequence of the previous statements.

Let now R_i denote the set of all cells reachable from the cell i ($\#R_i \leq 8$).

Given that they are the only possible moves, we have $\forall i, j$

$$p_{i,j} \Rightarrow \bigvee_{r \in R_i} p_{r,j+1} \leftrightarrow \neg p_{i,j} \bigvee \left(\bigvee_{r \in R_i} p_{r,j+1} \right)$$

These clauses are sufficient to solve the problem.

2 Question 2

My first attempt to encode the problem in a more efficient way was the following : let $s_{i,j}$ be the proposition "cell i is followed by cell j in the path". This encoding has the advantage that the value of most variables can be set to false : for each cell i, there are only at most 8 cells that are valid for $s_{i,j}$, the others can be set to false. Then, the solver will have many clauses that are just literals and will therefore be able to perform a large number of propagations very quickly.

However, I could not find a way to encode the fact that the knight must visit each cell. I therefore decided to use this new encoding together with the previous variables $p_{i,j}$, and to add clauses linking the two notions together. I also found that a few variables can have fixed values, and I removed all the redundant clauses : $(p \vee q) \wedge (q \vee p) \leftrightarrow (p \vee q)$.

With all these improvements, the solver is able to solve the problem for an 8×8 chessboard about twelve times faster than before.

3 Question 3

To force the model to compute a new model, only one additional clause is needed : the disjunction of the negations of all the position variables that were true in the previous model. This constraint means that **at least one of the cell un the path must be in a different position than before** and that's it.

4 Question 4

Previously, to obtain a new model, the added clause was the disjunction of the negations of all the position variables $p_{i,j}$ (cf. Question 1) that were true in the previous model. The idea remains the same, but for each symmetry, we add the disjunction of the variables $p_{i',j}$ where i' is the cell obtained by applying the symmetry to cell i.

5 Question 5

Since the solution is a path, one can visualize the multiple solutions for a path starting at (i_0, j_0) as a rooted tree.

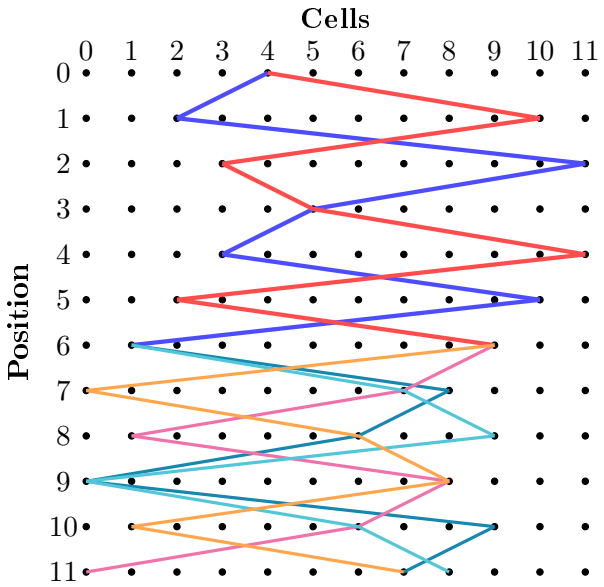

FIGURE 1 – Decision tree representing all possible paths for a 3×4 chessboard with $i_0 = 1$ and $j_0 = 0$

Intuitively, to obtain a list of constraints that ensures a unique solution, my algorithm enumerates all valid paths using the SAT solver and compares them iteratively. When several paths differ at a given position, one possible cell is chosen arbitrarily, and a corresponding constraint is added to keep only the consistent solutions. This process is repeated until only one path remains.

Note : One possible output of the algorithm (in the same situation as in Figure 1) is `[[1, 0, 1], [7, 1, 0]]` As can be seen in Figure 1, this indeed corresponds to a constraint that ensures a unique solution. (The cell $(0, 1)$ corresponds to column 1, and the cell $(1, 0)$ corresponds to column 4 in the figure.)